



Type Conversions & Promotions

→ There are two kinds of type conversions in Java :

1. Implicit

→ Conversions which are handled automatically by Java

→ For that, the destination data type must be wider than the source data type

byte b = 24;
✓ int num = b; → implicit casting to int by java, as int is wider than byte

→ This kind of conversion is called widening conversion

2. Explicit

→ When the destination data type is narrower than the source data type, java will not automatically convert / perform the cast

→ It must be handled explicitly by the developer by using cast

int i = 30;
X byte b = i; → compiler will give error

✓ byte b = (byte) i;
 └─────────> explicit cast

NOTE

↳ Even though casting will handle conversion, we still might face overflow / underflow issues

→ This kind of conversion is called narrowing conversion

Catch

`char c = 97;` → allowed, compiler will NOT give error

⇒ `c = 'a'`

Why?

→ Java internally stores unicode characters as integers only.

so, will this be allowed, `char c = 1_000_000;`?

⇒ NO! Here compiler will give an error!

→ Even though `char` allows direct integer input, there is a strict range we need to follow i.e. from 0 to 65,535

Truncating Conversion

→ In this scenario, when we explicitly convert a floating point number into an integer value, java truncates the floating part, storing only the integral part

float > int, long
double

`float f = 16.25;`

`int i = (int) f;` ⇒ `i = 16` → even though both `int` & `float` are of same size (32 bits)

NOTE

→ Boolean data type can not be casted to any other data type & vice-versa

Type Promotions

```
byte a = 20;
```

```
byte b = 4;
```

```
byte c = a * b;
```

should work right?

→ NO! This is because while performing arithmetic operations, java automatically promotes byte data type to int

Why?

→ Because while the arithmetic operation is being performed, what if the intermediary result exceeds the byte range!

```
byte a = 20; ✓
```

```
byte b = 40; ✓
```

```
byte c = 80; ✓
```

```
int calc = a * b + c
```

intermediary result is $20 \times 40 = 800$

which exceeds the byte range!

→ So to avoid incorrect calculations, java automatically promotes byte, short & char to int

More Generally, in any arithmetic operation all operands are widened into the data type of the operand with the highest data type.

```
↳ long l = 34;
```

```
int i = 200;
```

```
byte b = 12;
```

most wide (long)

$l * i + b$

widened into long